

Part A- Optimization

1.1 – f is convex

1.2 –

lets look at $g(u) = \begin{cases} 2 & x \geq 0 \\ 2x & x < 0 \end{cases}$

by the definition of $\partial f(u) \triangleq \{q \in V \mid \forall v \in V : f(v) \geq f(u) + q(v - u)\}$

now lets prove that $g(u) \in \partial f(u) \forall u \in \mathbb{R}$

1) $\forall u \geq 0 \Rightarrow g(u) = 2$ so $\forall v f(v) \geq f(u) + 2(v - u)$

1.1) if $v \geq 0$ we get $2v \geq 2u + 2(v - u) \Rightarrow 0 \geq 0$

1.2) if $v < 0$ we get $v^2 \geq 2u + 2(v - u) \Rightarrow v^2 \geq 2v \Rightarrow$ true for all $v < 0$

2) $\forall u < 0 \Rightarrow g(u) = 2u$ so $\forall v f(v) \geq f(u) + 2u(v - u)$

2.1) if $v \geq 0$ we get $2v \geq u^2 + 2u(v - u) \Rightarrow 2v \geq -u^2 + 2uv \Rightarrow$ true for all $u < 0$ and $v \geq$

0 since the left side of the inequality is positive and the left side is negative.

2.2) if $v < 0$ we get $v^2 \geq u^2 + 2(v - u) \Rightarrow v^2 \geq -u^2 + 2vu \Rightarrow$ true for all $u < 0$ and $v \geq 0$ since the left side of the inequality is positive and the left side is negative.

1.3 –

we update $\forall i \neq 0 \Rightarrow x_i = x_{i-1} - \eta \cdot \nabla f(x_{i-1}) = \begin{cases} \frac{x_{i-1}}{2} & x_{i-1} < 0 \\ x_{i-1} - 0.5 & x_{i-1} \geq 0 \end{cases}$

i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	$x_0 = -1.5$	$f(-1.5) = 2.25$	$g(-1.5) = -3$
1	$x_1 = -0.75$	$f(-0.75) = 0.5625$	$g(-0.75) = -1.5$
2	$x_2 = -0.375$	$f(-0.375) = 0.140625$	$g(-0.375) = -0.75$
3	$x_3 = -0.1875$	$f(-0.1875) = 0.03515625$	$g(-0.1875) = 0.375$

we can clearly see that $x_i \xrightarrow{i \rightarrow \infty} 0$ and because all the x_i values are negative

the function f will get a minimum at 0 (the function is $2x$)

so in the end the algorithm will converge to a minimum

1.4 –

we update $\forall i \neq 0 \Rightarrow x_i = x_{i-1} - \eta \cdot \nabla f(x_{i-1}) = \begin{cases} -x_{i-1} & x_{i-1} < 0 \\ x_{i-1} - 2 & x_{i-1} \geq 0 \end{cases}$

i	x_i	$f(x_i)$	$\frac{\partial}{\partial x} f(x_i) = g(x_i)$
0	$x_0 = -1.5$	$f(-1.5) = 2.25$	$g(-1.5) = -3$
1	$x_1 = 1.5$	$f(1.5) = 3$	$g(1.5) = 2$
2	$x_2 = -0.5$	$f(-0.5) = 0.25$	$g(-0.5) = -1$
3	$x_3 = 0.5$	$f(0.5) = 1$	$g(0.5) = 2$

we can clearly see that the algorithm is stuck in an infinte loop

in iteration 3 the s value is back the same as iteration 0

so the algorithm will never converge to a minimum

Part B – Regression

2.1 –

let $w_0 = 0, w_1 = 2, w_2 = 0, w_3 = 1$, the function then will be $f(x) = 2x + x^3$

$$f(0) = 2 \cdot 0 + 0 = 0$$

$$f(1) = 2 \cdot 1 + 1 = 3$$

$$f(2) = 2 \cdot 2 + 2^3 = 12$$

2.2 –

lets look at a quadratic polynomial $f(x) = ax^2 + bx + c$

to make the polynomial fit the data perfectly it should maintain 3 conditions:

$$f(0) = a \cdot 0 + b \cdot 0 + c = 0 \quad \Rightarrow \quad c = 0$$

$$f(1) = a \cdot 1 + b \cdot 1 + c = 3 \quad \Rightarrow \quad a + b = 3$$

$$f(2) = a \cdot 2^2 + b \cdot 2 + c = 12 \quad \Rightarrow \quad 4a + 2b = 12$$

$$a = 3, b = 0, c = 0$$

$$f(x) = 3x^2$$

2.3 –

$$\mathcal{L}(w) = \frac{1}{m} \|Xw - y\|_2^2$$

$$w_{2,1} = (0, 2, 0, 1)^T \Rightarrow \mathcal{L}(w_{2,1}) = \frac{1}{3} \left\| \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = 0$$

$$w_{2,2} = (0, 0, 3)^T \Rightarrow \mathcal{L}(w_{2,2}) = \frac{1}{3} \left\| \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = 0$$

2.4 –

$y = 6x - 1$ so the weight vector for the line is $w_{2,4} = (-1, 6)$

$$w_{2,4} = (-1,6) \Rightarrow \mathcal{L}(w_{2,4}) = \frac{1}{3} \left\| \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ 6 \end{pmatrix} - \begin{pmatrix} 0 \\ 3 \\ 12 \end{pmatrix} \right\|_2^2 = \frac{1}{3} \left\| \begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix} \right\|_2^2 = \frac{5}{3}$$

2.5 –

$$\mathcal{L}_\lambda(w) = \frac{1}{m} \|Xw - y\|_2^2 + \lambda \|w\|_2^2 = \mathcal{L}(w) + \lambda \|w\|_2^2$$

$$w_{2,1} = (0,2,0,1)^T \Rightarrow \mathcal{L}_\lambda(w_{2,1}) = \mathcal{L}(w_{2,1}) + \lambda \|w_{2,1}\|_2^2 = \lambda \left\| \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} \right\|_2^2 = 5\lambda$$

$$w_{2,2} = (0,0,3)^T \Rightarrow \mathcal{L}_\lambda(w_{2,2}) = \mathcal{L}(w_{2,2}) + \lambda \|w_{2,2}\|_2^2 = \lambda \left\| \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} \right\|_2^2 = 9\lambda$$

$$w_{2,1} = (-1,6)^T \Rightarrow \mathcal{L}_\lambda(w_{2,4}) = \mathcal{L}(w_{2,4}) + \lambda \|w_{2,4}\|_2^2 = \frac{5}{3} + \lambda \left\| \begin{pmatrix} -1 \\ 6 \end{pmatrix} \right\|_2^2 = \frac{5}{3} + 7\lambda$$

2.6 –

the $\mathcal{L}_\lambda(w)$ would always prefer the cubic polynomial over the quadratic one $5\lambda < 9\lambda$

this is because the way that we calculate the normalization is the L_2 norm and this way

as we saw multiple times in numerical algorithms the L_2 norm doesn't give penalty for bigger number of elements in a vector (bigger power).

2.7 –

by using a different norm for example L_1 norm which is defined by the sum of elements in the vector. thus,

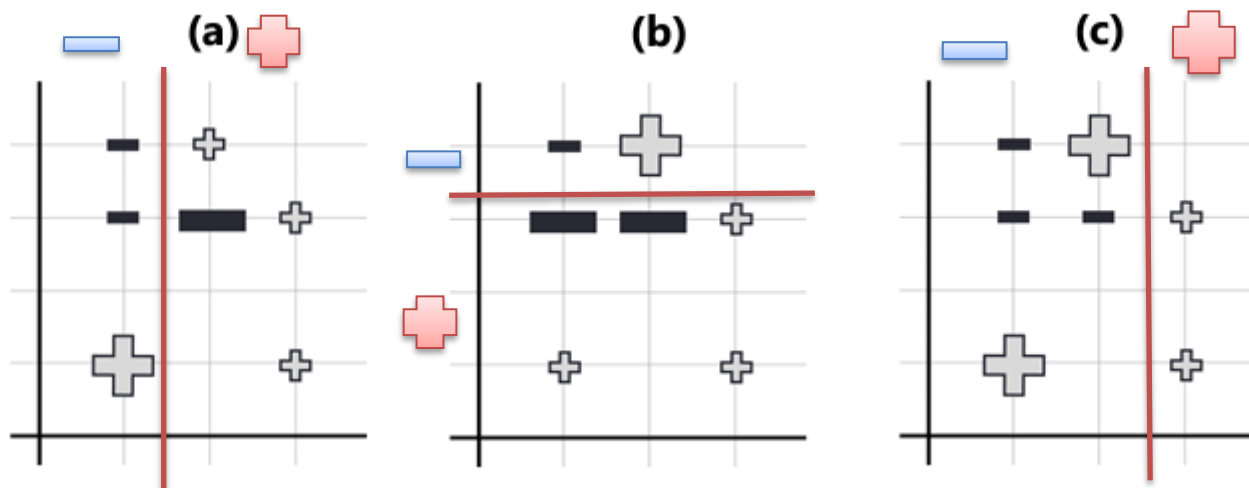
$$w_{2,1} = (0,2,0,1)^T \Rightarrow \mathcal{L}_\lambda(w_{2,1}) = \mathcal{L}(w_{2,1}) + \lambda \|w_{2,1}\|_1^2 = \lambda \left\| \begin{pmatrix} 0 \\ 2 \\ 0 \\ 1 \end{pmatrix} \right\|_1^2 = 9\lambda$$

$$w_{2.2} = (0,0,3)^T \Rightarrow \mathcal{L}_\lambda(w_{2.2}) = \mathcal{L}(w_{2.2}) + \lambda \|w_{2.2}\|_1^2 = \lambda \left\| \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix} \right\|_1^2 = 9\lambda$$

now we have the same value for the two weight vectors and we will choose the one with smaller degree.

Part C – Boosting

3 – only a, b and c



we can see in figure a that the all the data that are classified as minus and are to the left of the line got smaller so the classification of them is correct but the entry that is classified as plus and is on the left of the line got bigger also the entry that is classified as minus to the right of the line got bigger so its classification was wrong but the entries that are classified as plus and are to the right of the line got smaller so thier classification is right so our weak classifier will classify the points that are to the left of the line as minus and to the right of the line as plus.

in figure b the plus above the line was classifed wrong (got bigger) and the minus was right (got smaller) and under the line the plus got smaller (classified right) and the minus got bigger (classified right) so our weak classifier ill classify the points above the line as minus and under the line as plus.

in figure 3 the minus on the left of the line was classified right (got smaller) and plus was classifed right (got bigger) but on the right of the line the plus was classified right (got smaller) and the minus was classified wrong (got bigger). so our weak classifier will classify the points on the left of the line as minus and to the right of the line as plus

4 –

 $\forall t \in [T]$

$$\sum_{i=1}^m D_i^{t+1} \mathbb{I}_{[y_i \neq h_t(x_i)]} = 1/2.$$

Lets prove by induction on t :

base: ($t = 1$)

We have to prove that:

$$\sum_{i=1}^m D_i^2 \mathbb{I}_{[y_i \neq h_1(x_i)]} = \frac{1}{2}$$

let try to compute D_i^2 on some wrong classification on entry i with the assumption that there was n wrong classifications $0 < n < m$:

from adaboost algorithm the update of D_i^2 is:

$$D_i^2 \propto \frac{1}{m} e^{-\alpha_1 y_i h_1(x_i)} = \frac{1}{m} e^{\frac{1}{2} \ln(\frac{m}{n} - 1)} = \frac{1}{m} \sqrt{\frac{m}{n} - 1}$$



lets calculate α_1

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1}{\frac{n}{m}} - 1 \right) = \frac{1}{2} \ln \left(\frac{m}{n} - 1 \right)$$

let try to compute D_i^2 on some right classification on entry i with the assumption that there was n wrong classifications $0 < n < m$:

from adaboost algorithm the update of D_i^2 is:

$$D_i^2 \propto \frac{1}{m} e^{-\alpha_1 y_i h_1(x_i)} = \frac{1}{m} e^{-\frac{1}{2} \ln\left(\frac{m}{n}-1\right)} = \frac{1}{m} \sqrt{\frac{1}{\frac{m}{n}-1}} = \frac{1}{m} \sqrt{\frac{n}{m-n}}$$

now we have to normalize D_i^2 :

$$\text{normalization} = \sum_j D_j^1 e^{-\alpha_j y_i h_1(x_i)}$$

notice that the the sumed value has two options and it is determined by if the clasdification is right or wrong. thus,

$$\sum_j D_j^1 e^{-\alpha_j y_i h_1(x_i)} = n \left(\frac{1}{m} \sqrt{\frac{m}{n}-1} \right) + (m-n) \left(\frac{1}{m} \sqrt{\frac{n}{m-n}} \right) = \frac{n}{m} \sqrt{\frac{m-n}{n}} + \frac{m-n}{m} \sqrt{\frac{n}{m-n}} =$$

$$\frac{1}{m} \left(\sqrt{n(m-n)} + \sqrt{n(m-n)} \right) = \frac{2}{m} \sqrt{nm-n^2}$$

so for wrong classification:

$$D_i^2 = \frac{\frac{1}{m} \sqrt{\frac{m}{n}-1}}{\frac{2}{m} \sqrt{nm-n^2}} = \frac{1}{2} \cdot \frac{\sqrt{\frac{m}{n}-1}}{n \sqrt{\frac{m}{n}-1}} = \frac{1}{2n}$$

and for right calssification (we wont need it):

$$D_i^2 = \frac{\frac{1}{m} \sqrt{\frac{n}{m-n}}}{\frac{2}{m} \sqrt{nm-n^2}} = \frac{1}{2} \cdot \frac{\sqrt{\frac{n}{m-n}}}{n \sqrt{\frac{m}{n}-1}}$$

back to the original problem we now have to calulate:

$$\sum_{i=1}^m D_i^2 \mathbb{I}_{[y_i \neq h_1(x_i)]} = \sum_{i \in \{i | y_i \neq h_1(x_i)\}} D_i^2 = n D_i^{2 \text{ wrong classification}} = n \cdot \frac{1}{2n} = \frac{1}{2}$$

induction step:

$$\text{lets assume for k: } \sum_{i=1}^m D_i^{k+1} \mathbb{I}_{[y_i \neq h_k(x_i)]} = \frac{1}{2}$$

and lets prove for $k + 1$: $\sum_{i=1}^m D_i^{k+2} \mathbb{I}_{[y_i \neq h_{k+1}(x_i)]} = \frac{1}{2}$

firstly, lets assume that there is n wrong classifications $0 < n < m$

now to calculate D_i^{k+2} from the update rule in adaboost algorithm on some wrong classification i.e x_i – calculations are identical to base

$$D_i^{k+2} \propto D_i^{k+1} \sqrt{\frac{m}{n} - 1}$$

to calculate the normalization we have to calculate D_i^{k+2} on a right classification y_i

$$D_i^{k+2} \propto D_i^{k+1} \sqrt{\frac{n}{m-n}}$$

$$\begin{aligned} \text{normalization} &= \sum_j D_j^{k+1} e^{-\alpha_j y_i h_{k+1}(x_i)} = \sum_{j_{\text{wrong}}} \left(D_{j_{\text{wrong}}}^{k+1} \sqrt{\frac{m}{n} - 1} \right) + \sum_{j_{\text{right}}} \left(D_{j_{\text{right}}}^{k+1} \sqrt{\frac{n}{m-n}} \right) \\ &= \left(\sqrt{\frac{m}{n} - 1} \right) \cdot \left(\sum_{j_{\text{wrong}}} D_{j_{\text{wrong}}}^{k+1} \right) + \left(\sqrt{\frac{n}{m-n}} \right) \cdot \left(\sum_{j_{\text{right}}} D_{j_{\text{right}}}^{k+1} \right) = \\ &= \left(\sqrt{\frac{m}{n} - 1} \right) \cdot \left(\sum_{j_{\text{wrong}}} D_{j_{\text{wrong}}}^{k+1} \right) + \left(\sqrt{\frac{n}{m-n}} \right) \cdot \left(1 - \sum_{j_{\text{wrong}}} D_{j_{\text{wrong}}}^{k+1} \right) = \\ &= \frac{n}{m} \end{aligned}$$

after we add normalization to D_i^{k+2} for wrong classification we get:

$$D_i^{k+2} = \frac{D_i^{k+1} \sqrt{\frac{m}{n} - 1}}{\frac{n}{m}} =$$

$$\sum_{i=1}^m D_i^{k+2} \mathbb{I}_{[y_i \neq h_{k+1}(x_i)]} = \sum_{i \in \{i | y_i \neq h_{k+1}(x_i)\}} D_i^{k+2} = \sum_{i \in \{i | y_i \neq h_{k+1}(x_i)\}} \frac{D_i^{k+1} \sqrt{\frac{m}{n} - 1}}{\frac{n}{m}} = n \cdot \frac{1}{2n} = \frac{1}{2}$$

Part D – Perception

5 –

*in other words we have to prove that for $\eta_1 \neq \eta_2$
the algorithm will perform the same number of iterations,
and will converge to a vector that points to the same direction.*

let $\eta_1 \neq \eta_2 > 0$

lets prove by induction that the weight vector for both learning rates after each update is in the same direction (multiplied by positive constant)

base:

the algorithm will start with the same initial weight vector $w_0 = 0$.

so the value of $\hat{y}_i = \text{sign}(w^\top x_i)$ will be the same for both algorithms

so the first error will be at the same index i_1

the update will be for the weight vectors

$$w_{1_1} = w_{1_0} + \eta_1 y_{i_1} x_{i_1} = \eta_1 y_{i_1} x_{i_1}$$

$$w_{2_1} = w_{2_0} + \eta_2 y_{i_1} x_{i_1} = \eta_2 y_{i_1} x_{i_1}$$

which implies that both weight vectors are in the same direction

(and multiplied by positive constant because $\eta_1, \eta_2 > 0$)

$$\text{now } w_{1_1} = \frac{w_{2_1}}{\eta_1},$$

induction assumption:

$$w_{1_n} = \eta_1 \sum_{j \in \{1 \dots n\}} y_{i_j} x_{i_j}$$

$$w_{2_n} = \eta_2 \sum_{j \in \{1 \dots n\}} y_{i_j} x_{i_j}$$

induction step:

according to the induction assumption the difference between the prediction

and the actual label occurred on the same $x_{i_{n+1}}$

because the calculation of $\hat{y}_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i)$ will give the same answer in both cases
 thus, the update of the weight vectors will be

$$w_{1n+1} = w_{1n} + \eta_1 y_{i_{n+1}} x_{i_{n+1}} = \eta_1 \sum_{j \in \{1, \dots, n, n+1\}} y_{i_j} x_{i_j}$$

$$w_{2n+1} = w_{2n} + \eta_2 y_{i_{n+1}} x_{i_{n+1}} = \eta_2 \sum_{j \in \{1, \dots, n, n+1\}} y_{i_j} x_{i_j}$$

which implies that both weight vectors are in the same direction
 (and multiplied by positive constant because $\eta_1, \eta_2 > 0$).

conclusion:

the difference between y and $\hat{y}$ in both of the algorithms will always fall at the same x_i hence
 we have the same number of iterations until the algorithms stop
 also we proved that after each iteration the weight vectors point to the same direction,
 and this is true for the last iteration